README.md — Scrum Project Report Template

University of Cincinnati

EECE/CS-3093C — Software Engineering, Summer 2026

Instructor: Dr. Phu Phung

Scrum Project — TODO: Your Project Name

> Replace **"Messenger App Project"** with the title of your team's secure web application.

Team Members

Teams are 3–4 students (per syllabus). Solo teams are not permitted.

1. Member 1 Corey Brunner — brunnecj@mail.uc.edu — Scrum Master
2. Member 2 Pawan Poudyel — poudyepn@mail.uc.edu — Product Owner
3. Member 3 Zackary Bailey — baileyzc@mail.uc.edu — Developer
4. Member 4 Abdulah Dia — diaah@mail.uc.edu — Developer
5. Member 5 Garrett Becker — beckergt@mail.uc.edu — Developer

Project Management Information

Item	URL
Team homepage / landing page	https://github.com/uc-se-sm26-team1/uc-se-sm26-team1.github.io
Live prototype (Azure App Services)	https://TODO.azurewebsites.net
GitHub Projects board (private)	https://github.com/orgs/uc-se-sm26-team1/projects/2/views/1
Source code repository (private)	https://github.com/uc-se-sm26-team1/scrum-project
MongoDB Atlas cluster (configuration only — no credentials)	*e.g., cluster name, region*

Revision History

Date	Version	Description	Author
05/28/2028	0.1	Initial draft (Sprint 0) and README.md implementation	TODO
MM/DD/YYYY	0.2	Added use cases and architecture	TODO

Overview

Start in Sprint 0; refine across all sprints.

Describe the project in 2–4 paragraphs: the problem it addresses, the target users, and a high-level summary of the proposed solution. Include a **high-level architecture diagram**

System Analysis

Start in Sprint 0; keep updating.

User Requirements

List the high-level functional and non-functional requirements. These will be refined into user stories and use cases. *(Main focus of Sprint 0.)*

- **FR-1:** TODO — *As a [role], I want to [capability] so that [benefit].*
- **FR-2:** TODO
- **NFR-1 (Performance):** TODO
- **NFR-2 (Usability):** TODO
- **NFR-3 (Security — see §Security):** TODO

User Stories & Product Backlog

Link to your **GitHub Projects board** (above) and include a representative screenshot of the **Todo / In Progress / Done** columns at the end of each sprint. *(Sprint 0 onward.)*

Use Cases

Include the **use-case diagram** and a **brief description** (1–3 sentences) for each use case. *(Main focus of Sprint 0.)*

UC ID	Use Case	Primary Actor	Brief Description
UC-01	TODO	TODO	TODO
UC-02	TODO	TODO	TODO

System Design

Start in Sprint 1; keep updating.

Architecture

Describe the architectural style (e.g., layered, client-server, microservices) and the major components. Embed an architecture diagram if it differs from the high-level one in §Overview.

Use-Case Realization

For each use case in §Use Cases, describe how it is realized in code: which modules, endpoints, and database collections participate. **Sequence diagrams** are encouraged for non-trivial flows (e.g., authentication, message send/receive). *(Sprint 1 onward.)*

User Interface

Embed UI mockups or screenshots and describe the interaction model. Wireframes are acceptable for Sprint 1; final screenshots for Sprint 3. *(Sprint 1 onward.)*

Database

Describe your **MongoDB Atlas** schema: collections, fields, indexes, and relationships. Include a sample document for each collection. *(Sprint 2 onward; refine in Sprint 3.)*

```
// Example collection: users
{
  "_id": "ObjectId",
  "username": "string (unique, indexed)",
  "passwordHash": "string (bcrypt)",
  "createdAt": "ISODate"
}
```

Security (SSDLC)

*Start in Sprint 0; **mandatory** updates at the Sprint 1–2 SSDLC checkpoint and again in Sprint 3.*

This section documents how your team applies the **Secure Software Development Lifecycle** across every phase. Do **not** treat security as an afterthought — it is graded

across all sprints.

Security Requirements

List security requirements alongside functional requirements. *(Sprint 0.)*

- **SR-1:** TODO — *e.g., All authentication tokens must be transmitted over HTTPS only.*
- **SR-2:** TODO — *e.g., Passwords must be hashed with bcrypt (cost ≥ 12); plaintext passwords must never be logged or stored.*

Threat Model

Identify assets, trust boundaries, and threats. STRIDE or attack-tree format is acceptable. *(Sprint 0–1.)*

Asset	Threat	Mitigation
User credentials	Credential stuffing	Rate limiting + bcrypt
TODO	TODO	TODO

Security Review Notes

Summarize findings from your Sprint 2 security review and any remediation taken. *(Sprint 2 onward.)*

Implementation

Start in Sprint 1; keep updating.

Specify your development approach, languages, frameworks, and runtime. Default stack for this course:

Layer	Technology
Runtime	Node.js (Azure Cloud Shell for development)
Server framework	TODO *(e.g., Express)*
Database	MongoDB Atlas
Client	HTML / CSS / JavaScript *(framework optional)*
Version control	git + GitHub (branches + pull requests + code review)
Project mgmt	GitHub Projects
Hosting	Azure App Services
CI/CD	GitHub Actions
...	...

For each sprint, add a subsection that summarizes new implementation work. Include code snippets only when they illustrate a non-trivial design decision (not as a substitute for the source code itself).

Getting Started Locally

```
# Clone
git clone git@github.com:TODO/TODO.git
cd TODO

# Install dependencies
npm install

# Configure environment (copy and edit; never commit .env)
cp .env.example .env

# Run
npm start
```

CI/CD Pipeline

Describe the GitHub Actions workflow(s) under `.github/workflows/`. *(Sprint 1 onward.)*

- **Build & test:** triggered on every push and pull request.
- **Deploy:** triggered on merge to `main`; deploys to Azure App Services.

Deployment

Describe how to deploy and the URL of the live application. Include a note on environment variables (set in Azure App Services Configuration, never in source). *(Sprint 1 onward.)*

Testing & Quality Assurance

*Start in Sprint 1; **major** focus in Sprint 3.*

Test Plan

Summarize your testing strategy across unit, integration, and system testing. *(Sprint 2 onward.)*

Test Coverage

Report current test coverage and how to run the suite locally and in CI.

```
npm test
```

QA Plan

Manual test cases for user-facing flows, with expected vs. actual results. *(Sprint 3.)*

GenAI Usage & Reflection

*Start in Sprint 2; **required** in Sprint 3.*

Per the course academic integrity policy, the team must document all AI-assisted work on the team project. **Sprint 3 recommends the team to use a GenAI tool** for the final prototype and to document each substantive prompt.

Software Process Management

Start in Sprint 0; keep updating.

Describe how your team applies **Scrum**: roles, ceremonies (sprint planning, daily stand-ups, review, retrospective), and tools (GitHub Projects board, GitHub Issues, pull requests).

Include:

- A screenshot of the **GitHub Projects board** (Todo / In Progress / Done) at the end of each sprint.

- A **Roadmap view** screenshot from GitHub Projects, or a timeline produced from issue milestones. *(Note: GitHub Projects has a Roadmap view rather than a true Gantt chart; a Roadmap screenshot satisfies this requirement.)*

Scrum Process

> Copy the block below for each sprint (Sprint 0, 1, 2, 3).

Sprint X

Duration: YYYY-MM-DD to YYYY-MM-DD

Sprint Goal

TODO — one sentence.

Completed PBIs / Tasks

1. TODO
2. TODO
3. TODO

Contributions

Member	Hours	Contribution Summary
Member 1	X	TODO
Member 2	X	TODO
Member 3	X	TODO
Member 4	X	TODO
Member 5	X	TODO

Sprint Retrospective

Good	Could have been better	How to improve

Working through the sprints is a continuous-improvement process. The retrospective happens at the end of a sprint, before planning the next one. Cover three things briefly:

- **What went well** — celebrate and reinforce.
- **What could have been better** — be specific (e.g., "we underestimated authentication" not "things were hard").
- **How we will improve next sprint** — concrete, owned actions.

Keep it under an hour. The output is bullet points in the table above and any new PBIs created on the board.

User Guide / Demo

Start in Sprint 1; finalize in Sprint 3.

Write this section as both a **demo** (with screenshots of the running application) and a **how-to** for a first-time user. Cover sign-up, login, and the main user flows.

License & Code of Conduct

This project is developed for academic purposes as part of EECE/CS-3093C at the University of Cincinnati. The team follows the **ACM/IEEE Software Engineering Code of Ethics** (https://www.acm.org/code-of-ethics).

If your team chooses to publish the repository after the course, add an explicit license (e.g., MIT) here and a `LICENSE` file at the repo root.

End of template. Last template revision: TODO: YYYY-MM-DD.